به نام خدا

دانشگاه تهران

دانشکده مهندسی

برق و کامپیوتر

نام : مهسا ابرغانی اقدم

شماره دانشجویی :

درس NOC

نام استاد: دکتر مدرسی

In this CA, you should design a code to allocate TDM slots to a set of communication flows in a circuit-switched NoC with mesh topology. Given: --The network size --The number of slots in each slot table --The set of communication flows (source node, destination node, number of required slots in each window) The code should make a circuit for as many flows as possible. Circuits are made as a sequence of time slots from the source to the destination nodes. For this homework, you may design a novel circuit allocation algorithm or use existing ones.

برای حل این تمرین که هدف آن **اختصاص زمان‌بندی (Time Division Multiplexing) TDM** به مجموعه‌ای از جریان‌های ارتباطی در یک شبکه روی تراشه (NoC) با توپولوژی مش و مبتنی بر **سوئیچینگ مداری (Circuit-Switched)** است، باید الگوریتمی طراحی شود که بتواند بیشترین تعداد مدارها را با توجه به تعداد محدودی از **اسلات‌های زمانی** اختصاص دهد.

در ادامه چند الگوریتم پیشنهادی همراه با مزایا، معایب و نحوه عملکرد آورده شده‌اند:

1. Greedy TDM Slot Allocation

الگوریتم حریصانه (Greedy) یکی از ساده‌ترین الگوریتم‌هاست که تلاش می‌کند هر جریان را در اولین زمان ممکن به صورت کامل زمان‌بندی کند.

مراحل:

۱. لیست جریان‌ها را به ترتیب تعداد اسلات موردنیاز مرتب کن (مثلاً از کوچک به بزرگ).
۲. برای هر جریان:

مسیر بین مبدأ و مقصد را پیدا کن (مثلاً باXY Routing).

بررسی کن که آیا در یک پنجره زمانی خاص، تمام لینک‌های مسیر برای اسلات‌های موردنیاز آزاد هستند یا نه.

اگر آزاد بودند، آن اسلات‌ها را رزرو کن و مدار را ثبت کن.

مزایا: پیاده‌سازی ساده - مناسب برای شرایطی که تراکم کم است

معایب: ممکن است منابع را به شکلی غیربهینه مصرف کند و جریان‌های بلندتر جا نمانند - وابسته به ترتیب پردازش جریان‌ها

2. Maximal Matching + Time-Aware Routing

ایده این الگوریتم ترکیب دو مرحله‌ای از تطبیق بیشینه (در گراف) و بررسی امکان تخصیص زمانی است.

مراحل:ساخت گرافی که در آن یال‌ها جریان‌های ارتباطی هستند و گره‌ها مسیرها یا لینک‌های اشتراکی. با الگوریتم‌هایی مثل Hungarianیا Hopcroft–Karp حداکثر مجموعه‌ای از جریان‌هایی که به‌صورت هم‌زمان می‌توانند تخصیص داده شوند را انتخاب کن. برای این مجموعه، بررسی اسلات‌های آزاد در زمان‌بندی و تخصیص زمانی انجام بده.

مزایا: بهینه‌تر نسبت به الگوریتم حریصانه - مناسب برای تراکم‌های بالا

معایب: پیچیدگی زمانی بیشتر - نیاز به ساخت و نگهداری گراف

3. Simulated Annealing / Genetic Algorithm for Global Optimization

اگر هدف دستیابی به بهترین تعداد تخصیص با توجه به کل جریان‌هاست، می‌توان از الگوریتم‌های فرااکتشافی استفاده کرد.

ایده:

- یک کروموزوم = لیستی از اسلات‌های اختصاص‌یافته به هر جریان
- تابع هدف: بیشترین تعداد جریان‌های تخصیص‌یافته
- با جهش (mutation) و ترکیب (crossover) نسخه‌های جدید تولید شده و به مرور به پاسخ بهتر می‌رسیم.

مزایا: قابلیت یافتن راه‌حل نزدیک به بهینه - مناسب برای مسائل با فضای جستجوی بسیار بزرگ

معایب: زمان اجرا زیاد - نیاز به تنظیم پارامترهای زیاد

4. Time-Space Path Allocation (TSPA)

این الگوریتم به صورت سه‌بعدی (گره × مسیر × زمان) کار می‌کند و تلاش می‌کند یک مسیر در فضا-زمان برای هر جریان پیدا کند.

مراحل: برای هر جریان، نگاشت سه‌بعدی از لینک‌ها در زمان ساخته می‌شود. با الگوریتم‌های مسیریابی مانند Dijkstra در این گراف ۳بعدی، مسیر زمانی-مکانی مناسب پیدا می‌شود. اگر پیدا شد، اختصاص داده می‌شود، وگرنه رد می‌شود.

مزایا: دقیق - کنترل بهتر بر هم‌پوشانی زمانی مسیرها

معایب: حافظه و زمان پردازش زیاد

و اما روش آخر، روش ارائه شده در مقاله می باشد که به صورت زیر می باشد:

مرحله ۱: آغاز فرآیند تخصیص مدار از طریق گره مرکزی(Root)

در این الگوریتم، به‌جای آنکه هر گره به‌صورت مستقل مسیر مدار خود را جستجو کند (مانند روش‌های توزیع‌شده)، یک گره مرکزی به‌نام root مسئول بررسی و تخصیص مدارهاست. هر گره‌ای که نیاز به برقراری مدار دارد، درخواست خود را (شامل مبدأ، مقصد، زمان استفاده، و تعداد اسلات موردنیاز) از طریق یک شبکه کنترل سبک‌وزن و کم‌مصرف به root ارسال می‌کند. این تمرکز مرکزی باعث کاهش تراکم در شبکه اصلی و ساده‌سازی فرآیند تخصیص می‌شود.

مرحله ۲: جستجوی مسیر با الگوریتم BFS همراه با بررسی هم‌زمان اسلات‌های آزاد

پس از دریافت درخواست، الگوریتم در root آغاز به کار می‌کند و با استفاده از یک الگوریتم جستجوی سطح‌به‌سطح(BFS) ، مسیرهای حداقلی از مبدأ به مقصد را به‌صورت موازی بررسی می‌کند. در هر گره، وضعیت اشغال یا آزادی هر اسلات زمانی در قالب یک بردار بیتی (s_table) نگهداری می‌شود. برای بررسی تداوم زمانی در مسیر، یک بردار دیگر به‌نام free_slotsاز مبدأ آغاز می‌شود و در هر گره با اعمال عملیات منطقی) مانند AND و (OR و چرخش بیت‌ها بررسی می‌شود که آیا ادامه مسیر با رعایت تداوم زمانی ممکن است یا نه.

مرحله ۳: بررسی مقصد برای انتخاب اسلات‌های مناسب

زمانی که بردارهای free_slotsبه گره مقصد می‌رسند، سیستم بررسی می‌کند که آیا تعداد کافی اسلات زمانی آزاد و متوالی (در مسیرهای ممکن) وجود دارد یا نه. اگر چنین امکانی فراهم باشد، مقصد یک پیام تایید به سمت مبدأ ارسال می‌کند که شامل شماره اسلات‌هایی است که باید در طول مسیر رزرو شوند. این تاییدیه در ادامه مسیر بازگشتی به ترتیب در همه گره‌ها اعمال خواهد شد.

مرحله ۴: مسیر برگشتی (Backtracking) و رزرو نهایی اسلات‌ها

در مرحله بازگشت، با استفاده از اطلاعات بردار track_slotکه مسیر عبور داده‌ها را ذخیره کرده، مسیر برگشتی از مقصد به مبدأ طی می‌شود. در هر گره، بیت متناظر با اسلات رزرو شده در s_tableبه مقدار ۱ تغییر می‌یابد، که نشان‌دهنده اشغال آن اسلات برای این مدار است. همچنین شماره اسلات در مسیر برگشتی کاهش می‌یابد تا در هر گره، ترتیب زمانی صحیح حفظ شود.

مرحله ۵: به‌روزرسانی جدول‌های زمان‌بندی در همه گره‌ها

پس از پایان مراحل رزرو، rootنسخه نهایی جدول‌های زمان‌بندی را به گره‌های مسیر ارسال می‌کند. این جدول‌ها مشخص می‌کنند که در هر سیکل زمانی، هر گره باید از کدام ورودی به کدام خروجی اتصال برقرار کند. این فرآیند امکان انتقال داده‌ها را به‌صورت بدون تأخیر و بدون نیاز به بافر در طول مسیر فراهم می‌کند.

مرحله ۶: پشتیبانی از تخصیص مدار پیش‌بینی‌شده(Prediction-Based)

برای سیستم‌هایی که قابلیت پیش‌بینی زمان ارسال داده‌ها را دارند (مثلاً کنترلرهای حافظه یا پردازنده‌هایی با تحلیل کد)، الگوریتم امکان رزرو اسلات‌ها برای زمان‌های آینده را فراهم کرده است. در این حالت، rootچند جدول زمان‌بندی چرخشی برای بازه‌های زمانی آینده نگهداری می‌کند. درخواست‌هایی که مربوط به آینده باشند، اسلات خود را در جدول مناسب رزرو می‌کنند. این قابلیت باعث می‌شود مدارها در لحظه نیاز، بدون هیچ‌گونه تأخیر قابل استفاده باشند.

مرحله ۷: قابلیت تخصیص اسلات به چند مسیر(Multi-path)

اگر تعداد اسلات موردنیاز برای یک جریان داده بیش از ظرفیت یک مسیر منفرد باشد، الگوریتم می‌تواند بدون پیچیدگی اضافی، اسلات‌ها را بین چند مسیر مختلف تقسیم کند. این ویژگی با توجه به محدودیت منابع در شبکه‌های درون‌تراشه‌ای، باعث افزایش احتمال موفقیت در تخصیص مدار می‌شود، درحالی‌که همچنان تضمین می‌شود ترتیب زمانی داده‌ها حفظ شود.

مزایای اصلی الگوریتم

الگوریتم ارائه‌شده چندین مزیت مهم نسبت به روش‌های پیشین دارد. نخست، استفاده از یک گره مرکزی برای کنترل مسیر و اسلات‌ها باعث کاهش پیچیدگی، کاهش توان مصرفی، و حذف پیام‌های تکراری در شبکه می‌شود. دوم، استفاده از عملیات منطقی ساده) مانندAND ، OR، (rotateباعث می‌شود الگوریتم بسیار سبک و قابل پیاده‌سازی در سخت‌افزار با منابع بسیار محدود باشد. سوم، قابلیت پشتیبانی از مسیرهای چندگانه و تخصیص آینده‌نگرانه باعث افزایش کارایی و تطبیق بهتر با نیازهای سیستم‌های واقعی می‌شود. همچنین نتایج تجربی نشان داده‌اند این روش می‌تواند به‌طور متوسط تا ۲۷٪ تأخیر کمتر و ۲۱٪ مصرف انرژی کمتر نسبت به روش‌های کلاسیک داشته باشد.

معایب و چالش‌های الگوریتم

با وجود مزایای قابل‌توجه، این الگوریتم بی‌نقص نیست. نیاز به طراحی یک شبکه کنترل جداگانه میان گره‌ها و root یکی از چالش‌های اولیه است. در شبکه‌های بزرگ، rootمرکزی ممکن است به یک گلوگاه (bottleneck) تبدیل شود، هرچند روش ریشه‌های ناحیه‌ای (region-based roots) این مشکل را تا حدی حل می‌کند. همچنین اگر پیش‌بینی زمان ارسال داده‌ها نادرست باشد، اسلات‌های رزرو شده ممکن است بلااستفاده باقی بمانند. گرچه این اسلات‌ها می‌توانند توسط داده‌های-packet switchedاستفاده شوند، اما همچنان نوعی هدررفت منابع محسوب می‌شوند. در نهایت، نسبت به الگوریتم‌های ساده‌تر مانند Greedyیا DFS پیاده‌سازی آن پیچیده‌تر است و به هماهنگی دقیق‌تری بین سخت‌افزار و نرم‌افزار نیاز دارد.

مقدمه

در این تمرین مسأله از نوع تخصیص منابع در سطح زمان و مسیر در یک شبکه‌ی گره-به-گره است، جایی که جریان‌های داده‌ای نیاز دارند مدارهایی با مسیر اختصاصی از مبدأ تا مقصد خود در اختیار داشته باشند.

این مدارها باید نه‌تنها دارای اسلات‌های زمانی کافی باشند، بلکه با تأخیر مشخص در آینده نیز در جدول‌های TDM برقرار شوند. هدف این بود که بیشترین تعداد ممکن از این جریان‌ها تخصیص یابند.

کد نوشته شده و توضیحات آن:

```
import sys
import json
from collections import deque
import random
import uuid

# Directions:
DIRECTIONS = {'E': (1,0), 'W': (-1,0), 'N': (0,-1), 'S': (0,1)}
OPPOSITE = {'E':'W', 'W':'E', 'N':'S', 'S':'N'}

class ControlNetwork:
```

```
    """Simulates a lightweight control network for sending circuit
setup requests."""
    def __init__(self, width, height, root_node):
        self.width = width
        self.height = height
        self.root_node = root_node  # (x, y) coordinates of the root
node
        self.request_queue = deque()  # Queue for pending requests
        self.cycle_count = 0  # Tracks current cycle for latency
simulation
```

```
    def send_request(self, src, dst, slots, delay):
        """Simulate sending a request to the root node with latency."""
        # Calculate Manhattan distance to root (simplified latency model)
        latency = abs(src[0] - self.root_node[0]) + abs(src[1] - self.root_node[1]) + 1
        request = {'src': src, 'dst': dst, 'slots': slots, 'delay': delay, 'arrival_cycle': self.cycle_count + latency}
        self.request_queue.append(request)
        print(f"ControlNetwork: Request from {src} to {dst} queued, arrives at cycle {request['arrival_cycle']}")
        return latency

    def process_requests(self, root_controller, table_lifetime):
        """Process queued requests at the current cycle."""
        results = []
        while self.request_queue and self.request_queue[0]['arrival_cycle'] <= self.cycle_count:
            req = self.request_queue.popleft()
            print(f"ControlNetwork: Processing request from {req['src']} to {req['dst']}")
            res = root_controller.allocate(req['src'], req['dst'], req['slots'],
                                           self.cycle_count, req['delay'], table_lifetime)
            results.append((req, res))
        self.cycle_count += 1
        return results

    def get_packet_switched_slots(self, table_id, node, direction, root_controller):
        """Return available slots for packet-switched data."""
        return root_controller._free_mask(table_id, node, direction)

class RootController:
    def __init__(self, width, height, num_slots, num_tables=1, root_node=(0,0), hotspot_threshold=10):
        self.width = width
        self.height = height
        self.num_slots = num_slots
        self.num_tables = num_tables
        self.slot_mask = (1 << num_slots) - 1
        self.slot_tables = [
            {(x,y):{d:0 for d in DIRECTIONS}
             for x in range(width) for y in range(height)}
            for _ in range(num_tables)
        ]
        self.control_network = ControlNetwork(width, height, root_node)
        self.node_traffic = {(x,y):0 for x in range(width) for y in range(height)}
        self.hotspot_threshold = hotspot_threshold

    def validate_input(self, src, dst, slots, delay):
        """Validate input parameters."""
        if not (0 <= src[0] < self.width and 0 <= src[1] < self.height):
            raise ValueError(f"Invalid source coordinates: {src}")
        if not (0 <= dst[0] < self.width and 0 <= dst[1] < self.height):
            raise ValueError(f"Invalid destination coordinates: {dst}")
        if not (0 < slots <= self.num_slots):
            raise ValueError(f"Invalid number of slots: {slots}, must be between 1 and {self.num_slots}")
        if delay < 0:
            raise ValueError(f"Invalid delay: {delay}, must be non-negative")
        return True

    def allocate(self, src, dst, k, current_time, delay_cycles, table_lifetime):
        print(f"\nAllocating flow src={src}, dst={dst}, slots={k}, delay={delay_cycles}, current_time={current_time}")
        try:
            self.validate_input(src, dst, k, delay_cycles)
        except ValueError as e:
            print(f"Validation error: {e}")
            return None

        # Track traffic for hotspot detection
        self.node_traffic[dst] += k
        if self.node_traffic[dst] > self.hotspot_threshold:
            print(f"Warning: Node {dst} is a potential hotspot (traffic={self.node_traffic[dst]})")

        future = current_time + delay_cycles
        table_id = (future // table_lifetime) % self.num_tables
        print(f"Selected table_id={table_id} (future={future}, lifetime={table_lifetime})")
        n = self.num_slots
```

```
        # Initialize free and track
        free = {(x,y):0 for x,y in self.slot_tables[0]}
        track = {coord:{d:0 for d in DIRECTIONS} for coord in free}

        # BFS wave
        sx, sy = src
        dx, dy = dst
        mask0 = 0
        for d in ['E','W','N','S']:
            if self._delta_ok(d, sx, sy, dx, dy):
                m = self._free_mask(table_id, src, d)
                mask0 |= m
                track[src][d] |= m
        free[src] = mask0
        print(f"Initial free[src]={bin(free[src])}")
        q = deque([src])
        step = 0
        while q:
            u = q.popleft()
            print(f"Step {step}, pop node {u}, free={bin(free[u])}")
            merged_prev = free[u]
            for d in ['E','W','N','S']:
                if not self._delta_ok(d, u[0], u[1], dx, dy): continue
                v = (u[0] + DIRECTIONS[d][0], u[1] +
DIRECTIONS[d][1])
                if v not in free: continue
                pm = self._free_mask(table_id, u, d)
                merged = merged_prev & pm
                rot = ((merged << 1) & self.slot_mask) | ((merged >> (n-
1)) & 1)
                nm = self._free_mask(table_id, v, OPPOSITE[d])
                new = rot & nm & ~free[v]
                if new:
                    free[v] |= new
                    print(f"  Propagate to {v} dir={d}, new={bin(new)},
free[{v}]={bin(free[v])}")
                    for b in range(n):
                        if new & (1 << b): track[v][d] |= (1 << b)
                    q.append(v)
            step += 1
        avail = free[dst]
        print(f"Final available at dst {dst} = {bin(avail)}")
        if bin(avail).count('1') < k:
            print("  Not enough slots available, allocation fails.")
            return None

        # Random slot selection to balance load
        available_slots = [i for i in range(n) if avail & (1 << i)]
        bits = random.sample(available_slots, k) if
len(available_slots) >= k else []
        if not bits:
            print("  Not enough slots available after random selection,
allocation fails.")
            return None
        print(f"Selected slot bits: {bits}")

        # Backtracking and occupy
        results = []
        for b in bits:
            print(f"Backtracking slot bit={b}")
            path = []
            cur = dst
            bit = b
            while cur != src:
                for d, mask in track[cur].items():
                    if mask & (1 << bit):
                        prev = (cur[0] - DIRECTIONS[d][0], cur[1] -
DIRECTIONS[d][1])
                        path.append((prev, d))
                        print(f"  at {cur}, via {d} to {prev}, bit now {(bit-
1)%n}")
                        bit = (bit - 1) % n
                        cur = prev
                        break
            path.reverse()
            print(f"  Path for bit {b}: {path}")
            cur = src
            depth = 0
            for _, d in path:
                idx = (b + depth) % n
                print(f"  Occupy at {cur}->{d}, slot {idx}")
                self._occupy(table_id, cur, d, idx)
                nxt = (cur[0] + DIRECTIONS[d][0], cur[1] +
DIRECTIONS[d][1])
                self._occupy(table_id, nxt, OPPOSITE[d], idx)
                cur, depth = nxt, depth + 1
            results.append((table_id, b, path))
        print(f"Allocation result: {results}")

        # Report available slots for packet-switched data
```

```
        for node in free:
            for d in DIRECTIONS:
                free_slots = self._free_mask(table_id, node, d)
                if free_slots:
                    print(f"Packet-switched slots available at {node}->{d}: {bin(free_slots)}")

        return results

    def _free_mask(self, tid, coord, d):
        return ~self.slot_tables[tid][coord][d] & self.slot_mask

    def _occupy(self, tid, coord, d, slot):
        self.slot_tables[tid][coord][d] |= (1 << slot)

    def _delta_ok(self, d, sx, sy, dx, dy):
        if d == 'E': return dx > sx and sx < self.width - 1
        if d == 'W': return dx < sx and sx > 0
        if d == 'S': return dy > sy and sy < self.height - 1
        if d == 'N': return dy < sy and sy > 0
        return False

def generate_random_flows(width, height, num_slots, max_flows, max_delay):
    """Generate random flows within specified ranges."""
    num_flows = random.randint(1, max_flows)
    flows = []
    for _ in range(num_flows):
        src = [random.randint(0, width-1), random.randint(0, height-1)]
        dst = [random.randint(0, width-1), random.randint(0, height-1)]
        while src == dst:  # Ensure src != dst
            dst = [random.randint(0, width-1), random.randint(0, height-1)]
        slots = random.randint(1, num_slots)
        delay = random.randint(0, max_delay)
        flows.append({"src": src, "dst": dst, "slots": slots, "delay": delay})
    return flows

def save_config_to_file(config, filename="generated_flows.json"):
    """Save configuration with random flows to a JSON file."""
    try:
        with open(filename, 'w') as f:
            json.dump(config, f, indent=4)
        print(f"Configuration saved to {filename}")
    except IOError as e:
        print(f"Error saving configuration: {e}")
        sys.exit(1)

def save_results_to_file(successful_flows, failed_flows, filename="allocation_results.json"):
    """Save allocation results to a JSON file."""
    results = {
        "successful_flows": [
            {"flow_id": flow_id, "allocation": [
                {"table_id": r[0], "slot": r[1], "path": r[2]} for r in res
            ]} for flow_id, res in successful_flows
        ],
        "failed_flows": failed_flows
    }
    try:
        with open(filename, 'w') as f:
            json.dump(results, f, indent=4)
        print(f"Results saved to {filename}")
    except IOError as e:
        print(f"Error saving results: {e}")
        sys.exit(1)

if __name__ == '__main__':
    # Default configuration
    default_config = {
        "width": 4,
        "height": 4,
        "num_slots": 8,
        "num_tables": 2,
        "table_lifetime": 50,
        "root_node": [0, 0],
        "max_flows": 10,
        "max_delay": 1000,
        "hotspot_threshold": 10
    }

    # Generate random flows if no input file is provided
    if len(sys.argv) == 2:
        try:
            with open(sys.argv[1]) as f:
                cfg = json.load(f)
            print(f"Configuration loaded from {sys.argv[1]}")
```

```
        except (IOError, json.JSONDecodeError) as e:
            print(f"Error reading input file: {e}")
            sys.exit(1)
    else:
        print("No flows file provided, generating random flows.")
        cfg = default_config.copy()
        cfg['flows'] = generate_random_flows(
            cfg['width'], cfg['height'], cfg['num_slots'],
            cfg['max_flows'], cfg['max_delay']
        )
        save_config_to_file(cfg)

    # Validate configuration
    try:
        if not (cfg['width'] > 0 and cfg['height'] > 0):
            raise ValueError("Invalid network dimensions")
        if not (cfg['num_slots'] > 0):
            raise ValueError("Invalid number of slots")
        if not (cfg['num_tables'] > 0):
            raise ValueError("Invalid number of tables")
        if not (cfg['table_lifetime'] > 0):
            raise ValueError("Invalid table lifetime")
        if not (0 <= cfg['root_node'][0] < cfg['width'] and 0 <=
cfg['root_node'][1] < cfg['height']):
            raise ValueError(f"Invalid root node coordinates:
{cfg['root_node']}")
        if not (0 < cfg.get('max_flows', 1) <= 100):
            raise ValueError(f"Invalid max_flows:
{cfg.get('max_flows')}")
        if not (0 <= cfg.get('max_delay', 0) <= 10000):
            raise ValueError(f"Invalid max_delay:
{cfg.get('max_delay')}")
        if not (cfg.get('hotspot_threshold', 1) > 0):
            raise ValueError(f"Invalid hotspot_threshold:
{cfg.get('hotspot_threshold')}")
    except KeyError as e:
        print(f"Configuration error: Missing key {e}")
        sys.exit(1)
    except ValueError as e:
        print(f"Configuration error: {e}")
        sys.exit(1)

    print(f"Configuration: width={cfg['width']},
height={cfg['height']}, slots={cfg['num_slots']}, "
          f"tables={cfg.get('num_tables',1)},
lifetime={cfg.get('table_lifetime',1)}, "
          f"root_node={cfg.get('root_node')},
max_flows={cfg.get('max_flows')}, "
          f"max_delay={cfg.get('max_delay')},
hotspot_threshold={cfg.get('hotspot_threshold')}")

    root = RootController(cfg['width'], cfg['height'],
cfg['num_slots'],
                          cfg.get('num_tables', 1), cfg.get('root_node', (0,0)),
                          cfg.get('hotspot_threshold', 10))
    tl = cfg.get('table_lifetime', 1)
    flows = cfg['flows']

    # Sort flows by number of slots to maximize allocations
    flows = sorted(flows, key=lambda x: x['slots'])
    print(f"Processing {len(flows)} flows (sorted by slots)...")

    succ = []
    fail = []
    for i, flow in enumerate(flows):
        try:
            src = tuple(flow['src'])
            dst = tuple(flow['dst'])
            k = flow['slots']
            d = flow.get('delay', 0)
            print(f"=== Flow {i}: src={src} dst={dst} slots={k}
delay={d} ===")
            root.control_network.send_request(src, dst, k, d)
        except KeyError as e:
            print(f"Flow {i} error: Missing key {e}")
            fail.append(i)
            continue
        except ValueError as e:
            print(f"Flow {i} error: {e}")
            fail.append(i)
            continue

    # Process requests for multiple cycles
    max_cycles = 20
    for cycle in range(max_cycles):
        print(f"\n=== Processing cycle {cycle} ===")
        results = root.control_network.process_requests(root, tl)
        for req, res in results:
```

```
        flow_id = next((i for i, f in enumerate(flows) if f['src'] ==
list(req['src']) and
                       f['dst'] == list(req['dst']) and f['slots'] ==
req['slots'] and
                       f.get('delay', 0) == req['delay']), -1)
        if res:
            succ.append((flow_id, res))
        else:
            fail.append(flow_id)
```

```
print(f"Summary: Allocated {len(succ)}/{len(flows)} flows.")
if succ:
    for i, res in succ:
        print(f"  Flow {i} result: {res}")
if fail:
    print(f"  Failed flows: {fail}")

# Save results to file
save_results_to_file(succ, fail)
```

ساختار و منطق کلی کد

۱ .شبیه‌سازی ساختار شبکه و جدول اسلات‌ها

برای مدل‌سازی شبکه، از ساختار دوبعدی width × height استفاده کردم، به‌طوری که هر گره دارای چهار جهت ارتباطی (شرق، غرب، شمال، جنوب) است. برای هر جهت در هر گره، جدولی از اسلات‌های زمانی TDM طراحی شده که با یک عدد صحیح و ماسک بیتی پیاده‌سازی شده‌اند.

مثلاً در صورتی که ۸ اسلات داشته باشیم، هر ماسک ۸ بیت دارد و مقدار 0نشان‌دهنده‌ی اسلات آزاد است.

مزیت ماسک بیتی:
سرعت بسیار بالا در بررسی، ترکیب و اشغال اسلات‌ها و صرفه‌جویی در حافظه نسبت به استفاده از لیست یا ماتریس‌های مجزا.

۲ .طراحی شبکه کنترلی برای مدیریت درخواست‌ها

به منظور شبیه‌سازی تأخیرات کنترلی و ارسال پیام‌های برقراری مدار، یک شبکه‌ی کنترل طراحی کردم (ControlNetwork) که نقش آن ارسال درخواست‌های جریان‌ها به ریشه‌ی شبکه و دریافت پاسخ است.

درخواست‌ها با توجه به فاصله‌ی منهتنی تا ریشه (گره ۰،۰) دارای تأخیر متفاوتی هستند و در صف قرار می‌گیرند. این ساختار باعث می‌شود که رفتار سیستم در چرخه‌های زمانی نیز قابل کنترل و واقع‌گرایانه باشد.

۳ .الگوریتم مسیریابی و تخصیص اسلات

برای اختصاص اسلات‌ها از نسخه‌ای هوشمند از BFS استفاده شده است. روند به این شکل است:

از گره مبدأ شروع می‌کنیم.

در هر گره، اسلات‌های آزاد در هر جهت بررسی می‌شوند و تنها در صورت وجود مسیر آزاد، گسترش ادامه می‌یابد.

ساختار freeبرای نگهداری ماسک‌های اسلات آزاد و trackبرای ثبت حرکت‌ها استفاده می‌شود.

پس از رسیدن به مقصد، مسیر با استفاده از اطلاعات trackبه صورت Backtracking بازسازی می‌شود و در هر مرحله، اسلات مناسب در جهت مناسب اشغال می‌شود.

نکته مهم: در هر مرحله، اگر تعداد اسلات‌های آزاد کافی نباشد، جریان رد شده و در لیست ناموفق‌ها قرار می‌گیرد.

نحوه‌ی تولید و مدیریت جریان‌ها(Flows)

در صورتی که فایل ورودی برای جریان‌ها ارائه نشده باشد، سیستم به صورت تصادفی تعدادی جریان تولید می‌کند که شامل موارد زیر هستند:

- مختصات گره مبدأ
- مختصات گره مقصد (غیر مساوی با مبدأ)
- تعداد اسلات مورد نیاز
- تأخیر مورد انتظار تا برقراری مدار

جریان‌ها پس از تولید، به ترتیب صعودی بر اساس تعداد اسلات مرتب می‌شوند تا جریان‌های سبک‌تر زودتر تخصیص یابند و درصد موفقیت بالا رود.

تحلیل خروجی برنامه

بر اساس اجرای واقعی برنامه با پیکربندی پیش‌فرض:

- ابعاد شبکه: ۴ × ۴
- تعداد اسلات: ۸
- تعداد جدول: ۲
- عمر هر جدول: ۵۰ چرخه
- حداکثر تأخیر: ۱۰۰۰
- حداکثر جریان تصادفی: ۱۰

نتایج کلی:

- تعداد جریان تولید شده: ۵
- تعداد جریان‌های موفق: ۵ (یعنی ۱۰۰٪)

این نشان می‌دهد که الگوریتم توانسته در شرایط مختلف، به‌طور کامل مسیر و اسلات مناسب برای تمام جریان‌ها پیدا کند و تخصیص دهد.

نمونه تحلیل مسیر تخصیص:

برای جریان شماره ۰ با مشخصات:

src=(1,1), dst=(2,0), slots=1, delay=823

مسیر یافته‌شده:

(1,1) → E → (2,1) → N → (2,0)

اسلات انتخاب‌شده: اسلات شماره ۴

هر لینک در این مسیر به صورت جداگانه و در جهت صحیح اشغال شده است.

هشدار:Hotspot

در جریان شماره ۴، گره مقصد (3,1)چندین بار مقصد جریان‌ها قرار گرفت. برنامه به‌درستی این گره را به‌عنوان نقطه‌ی داغ (hotspot)شناسایی کرد و پیام هشداری چاپ کرد. این یعنی سیستم فقط تخصیص نمی‌دهد بلکه ترافیک و ازدحام را هم پایش می‌کند.

ذخیره‌سازی نتایج:

در پایان اجرای موفق، تمام مسیرها و اطلاعات تخصیص‌شده در فایل allocation_results.jsonذخیره می‌شوند. هر جریان موفق دارای اطلاعات زیر است:

- شماره جدول تخصیص‌یافته
- شماره اسلات استفاده‌شده
- مسیر اختصاص‌یافته به هر اسلات

روش‌ها و دلایل فنی انتخاب‌شده

بخش	روش استفاده‌شده	دلیل انتخاب
تخصیص اسلات	BFS + Backtracking	تضمین مسیر کامل، حفظ اسلات‌های پیوسته، حافظه کم
انتخاب اسلات‌ها	انتخاب تصادفی از بین آزادها	توزیع یکنواخت بار روی جدول
نمایش وضعیت	ماسک بیتی(**int** ops)	پردازش سریع، مصرف حافظه کم
تأخیر کنترل	فاصله منهتنی تا ریشه	شبیه‌سازی ساده و واقع‌گرایانه زمان
پایش بار	شمارش ترافیک هر گره	شناسایی ازدحام و نقاط گلوگاه

جمع‌بندی نهایی

الگوریتم من توانست در یک محیط شبکه‌ای محدود و زمان‌بندی‌شده، جریان‌هایی با نیازهای متفاوت را به درستی تخصیص دهد. طراحی مبتنی بر الگوریتم BFS با انتخاب‌های تصادفی اسلات و مدل تأخیری در شبکه کنترل باعث شد تا هم عملکرد و هم پوشش جریان‌ها بهینه باشند.